

Reviewer Pack — Minimal Order of Diophantine Equations and Resolution Proof ...

Entry 6ddb6e0cdb5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 05:05:57 UTC

Minimal Order of Diophantine Equations and Resolution Proof Width

Entry ID: 6ddb6e0cdb5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 05:05:57 UTC

1. Conjecture

Field A (mathematical branch): Number Theory (Diophantine Equations) **Field B** (complexity object): Resolution Proofs (Proof Complexity)

Statement:

The minimal order of a diophantine equation associated with a Boolean satisfiability instance φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $O(n^2) \leq O(w(\varphi))$ where n is the number of variables in φ .

Rationale (proposer's reasoning):

Diophantine equations can encode computational complexity, and their solution may reveal structural information about the complexity of satisfiability. The minimal order could provide a new insight into the proof width by offering a direct link to number-theoretic properties.

Taxonomy category: DIOPHANTINE_EQNS_RESOLUTION_WIDTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): cd1cba6603bd8629

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean resolution proof width for all generated CNF formulas exceeds n^2 , where n is the number of variables in the formula, and no seed produces a resolution proof width less than n^2 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -diophantine equation AND resolution proof complexity - resolution proof width AND minimal order diophantine equations - Boolean satisfiability instance AND $w(\phi)$ linearly correlated with $O(n^2)$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(10 * n): # Ensure at least 10 clauses per variable
            clause = [random.randint(-n, -1), random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def diophantine_equation(cnf):
        n = max(abs(lit) for lit in sum(cnf, []))
        A = [[0] * (2 * n + 1) for _ in range(len(cnf))]
        b = [0] * len(cnf)

        for i, clause in enumerate(cnf):
            for lit in clause:
                if lit > 0:
                    row = i
                    col = lit - 1
                else:
                    row = i
                    col = -(lit + 1) + n

                A[row][col] += 1

        return A, b

    def gaussian_elimination(A, b):
        m, n = len(A), len(A[0])
```

```

for i in range(m):
    max_row = i
    for j in range(i+1, m):
        if abs(A[j][i]) > abs(A[max_row][i]):
            max_row = j
    A[i], A[max_row] = A[max_row], A[i]
    b[i], b[max_row] = b[max_row], b[i]

    for j in range(i+1, m):
        factor = A[j][i] / A[i][i]
        for k in range(i, n):
            A[j][k] -= factor * A[i][k]
        b[j] -= factor * b[i]

x = [0] * n
for i in range(m-1, -1, -1):
    x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]

return x

def resolution_width(cnf):
    clauses = set(tuple clause for clause in cnf)
    queue = list(clauses)
    width = 0

    while queue:
        clause = queue.pop()
        if len(clause) > width:
            width = len(clause)

        new_clauses = []
        for other_clause in clauses:
            if not set(clause).isdisjoint(other_clause):
                for lit1 in clause:
                    for lit2 in other_clause:
                        if abs(lit1) == abs(lit2):
                            continue
                        new_lit = -lit1 if lit1 < 0 else -lit2
                        new_clause = tuple(sorted(set(clause + other_clause) - {new_lit}))
                        if new_clause not in clauses and new_clause not in new_clauses:
                            new_clauses.append(new_clause)
        queue.extend(new_clauses)
        clauses.update(new_clauses)

    return width

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    cnf = generate_cnf(n)
    A, b = diophantine_equation(cnf)
    x = gaussian_elimination(A, b)

    if any(math.isnan(val) or math.isinf(val) for val in x):
        continue

```

```

w = resolution_width(cnf)
results.append({"n": n, "order": len(x), "width": w})

if not results:
    return {
        "metric_name": "resolution_width",
        "metric_value": 0,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "no_valid_instances"
    }

total_order = sum(result["order"] for result in results)
total_width = sum(result["width"] for result in results)
instances_tested = len(results)
n_max = max(result["n"] for result in results)

mean_order = total_order / instances_tested
mean_width = total_width / instances_tested

if all(mean_width >= n**2 for n in [5, 10, 15, 20, 30, 40]):
    return {
        "metric_name": "resolution_width",
        "metric_value": mean_order,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": True,
        "counterexample": ""
    }
else:
    for result in results:
        if result["width"] < result["n"]**2:
            return {
                "metric_name": "resolution_width",
                "metric_value": mean_order,
                "instances_tested": instances_tested,
                "n_max": n_max,
                "conjecture_holds": False,
                "counterexample": f"width<{result['n']}^2 for n={result['n']}"
            }

    return {
        "metric_name": "resolution_width",
        "metric_value": mean_order,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "unknown"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(res["metric_value"] for res in results) / len(results)
support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

if all(res["conjecture_holds"] for res in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction=1")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='first_failing_seed' first_failing_seed={first_fa

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_879d1d4e.py", line 162, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_879d1d4e.py", line 100, in run_trial
    x = gaussian_elimination(A, b)
        ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_879d1d4e.py", line 57, in gaussian_elim
    factor = A[j][i] / A[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to a division by zero error before producing data that could confirm or falsify the conjecture. | next: Investigate and fix the division by zero error in the code, then rerun the test with a larger sample size to ensure robustness of the results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13585
2	preregistration	ollama_remote	glm4:latest	0	0	9436
3	novelty	ollama_remote	glm4:latest	0	0	8123
4	novelty	ollama_remote	glm4:latest	0	0	9568
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18911
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7185
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13187
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17963
9	judge	ollama_remote	glm4:latest	0	0	11829

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 109786 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/6ddbe6e0cdb5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6ddbe6e0cdb5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6ddbe6e0cdb5.tar.gz` (if generated)